Bluetooth Detective: Final Report

Jessica Forrett
Ivan Martinovic
Anna Cherkinsky

Future Changes to the Physical Prototype:

After our usability testing sessions, it was clear that carrying a large box attached to the device causes usability concerns. The suggestions from users to make our device smaller is not an option because the basic principle which allows the device to work requires considerable physical separation between the Bluetooth modules for the location estimates to be as precise as possible. A better redesign would be to make the container for the interior components, such as the Bluetooth modules, larger. The container would then be stationary, and would simply need to be placed in the proximity of where the user believes the Bluetooth devices they are searching for are located. The larger dimensions for the device container will allow for a wider distance between Bluetooth modules, which will increase precision of the search for the devices. The user would then be able to simply use the BluetoothDetective app on their phone (or other mobile device) to search for Bluetooth devices in camera view, without having to hold a large, heavy container alongside their phone.

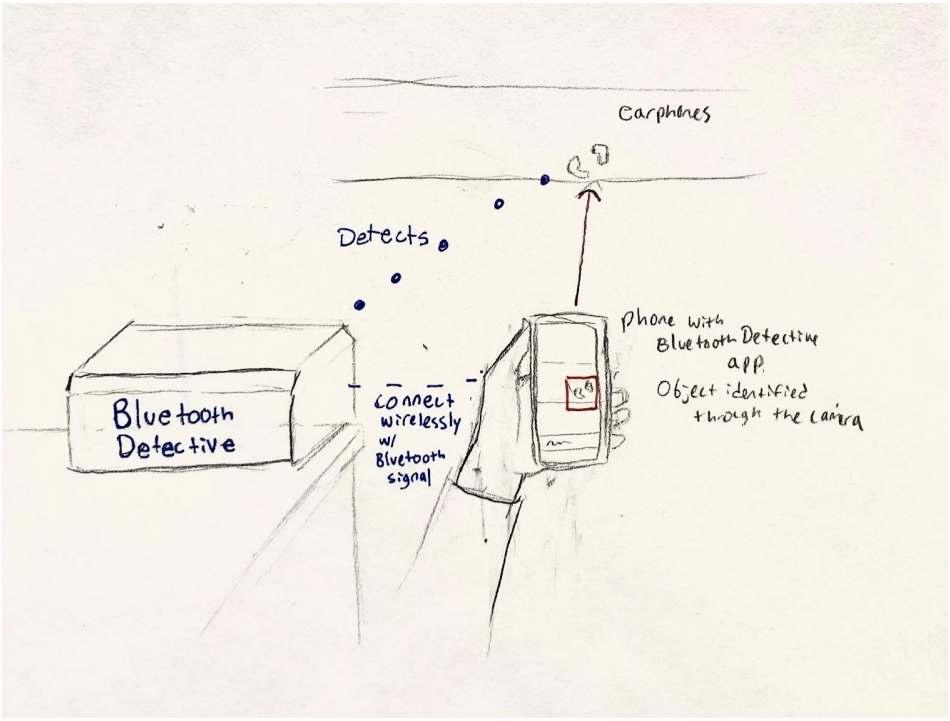


Figure 1: The BluetoothDetective is larger, placed on a surface. It is wirelessly connected to the device with the BluetoothDetective app (phone, in this case). It detects the earphones, which are identified on the phone.

The Updated Digital Prototype:

Some improvements we made were to add some basic instructions to the home and loading screens, since some of our test users were a little confused about what they should do. We also made some additions to the camera view to aid users and address some issues that came up during testing.

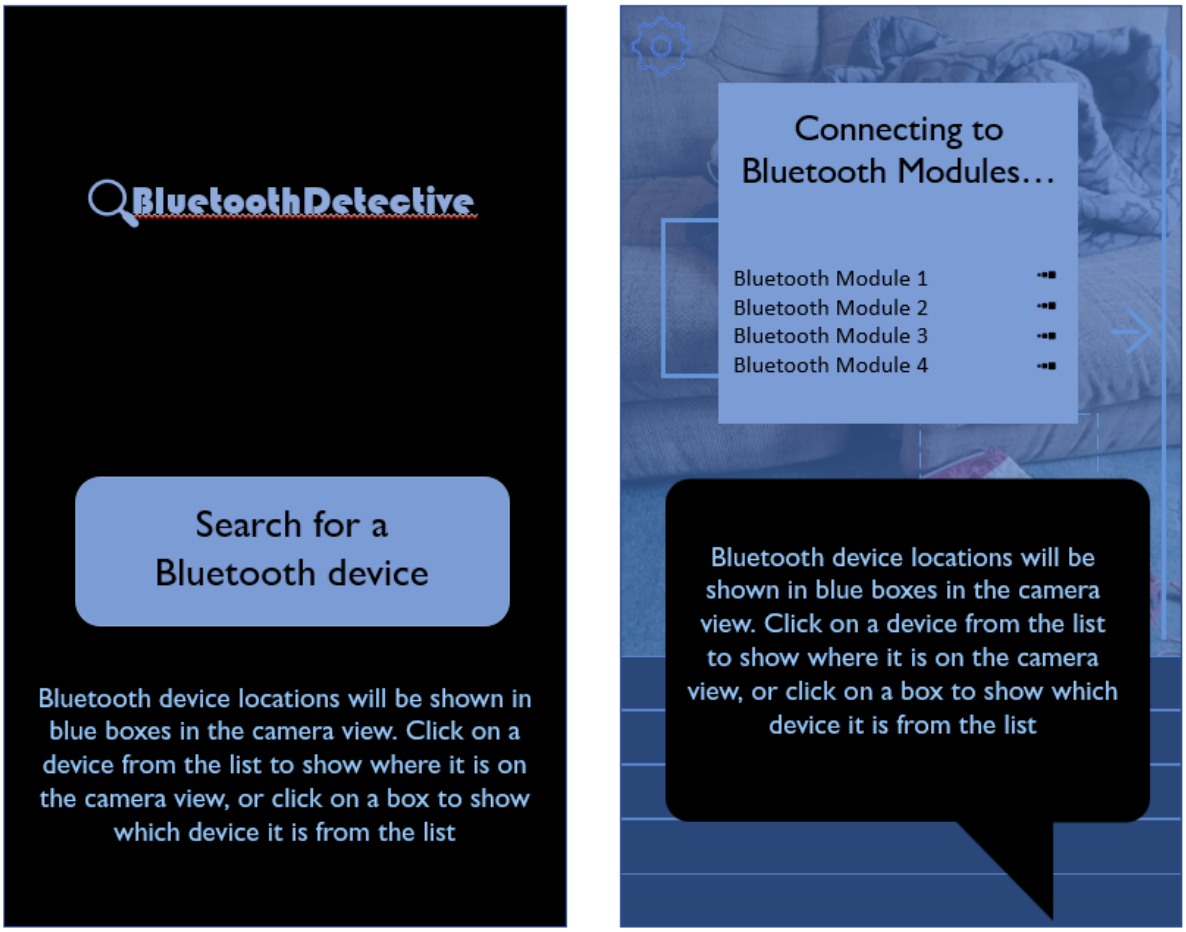
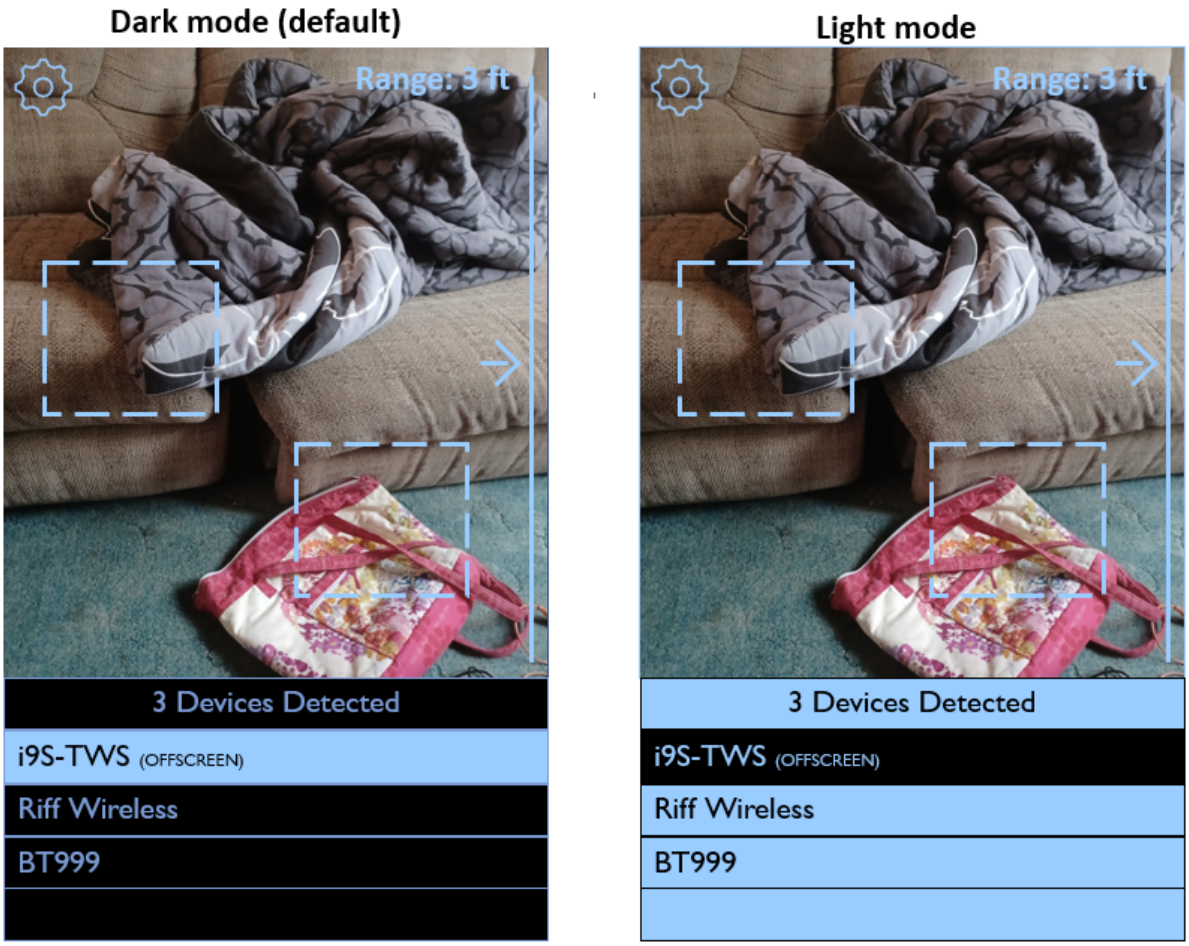


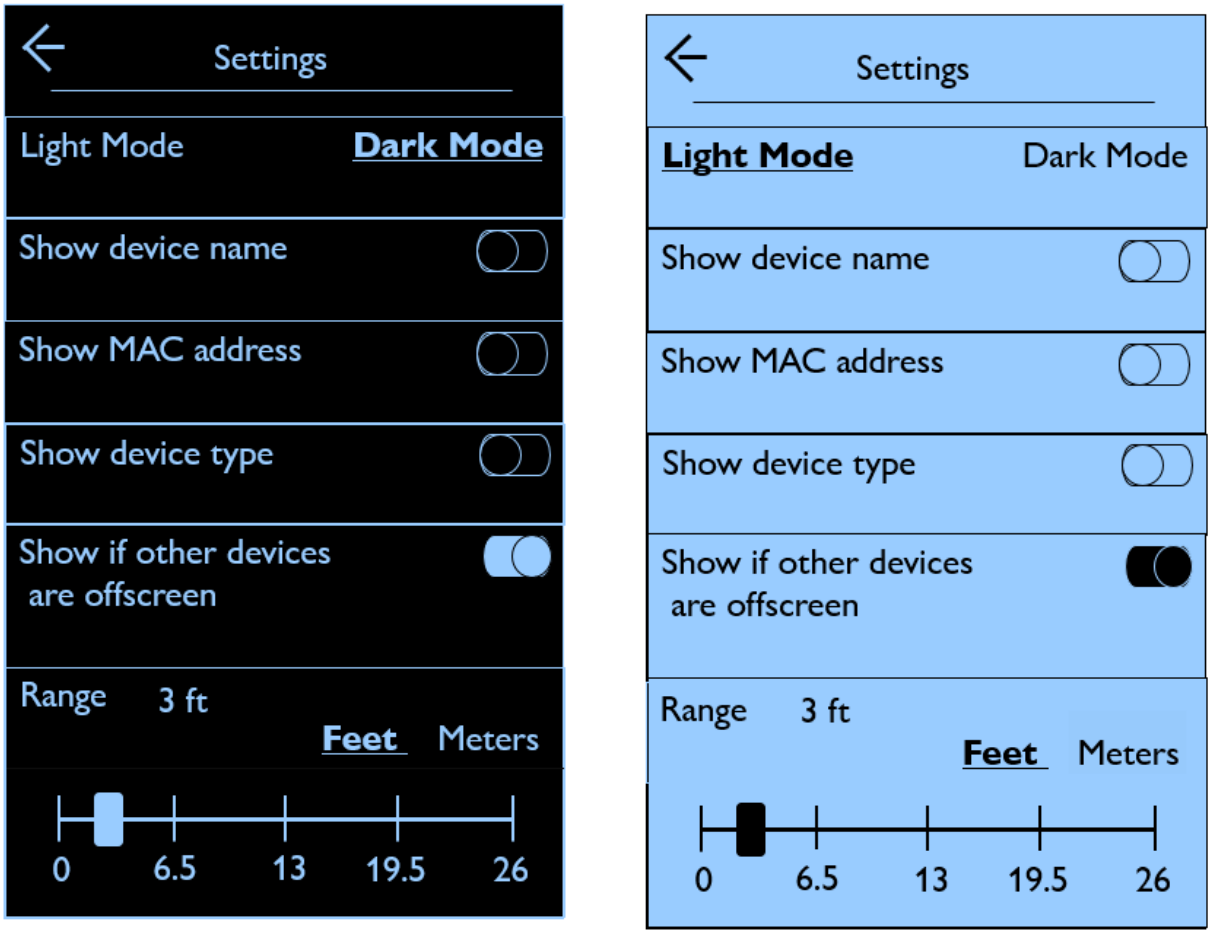
Figure 2 and **3** The left image depicts the start screen, and right is the loading screen

For example, the "show offscreen bluetooth devices" option is now turned on by default, and devices that are offscreen are now labeled as such. Another addition is the ability to click on a square to highlight that square, as well as the corresponding bluetooth device. We made this change because users expected that clicking on the squares would do something. We also added the range of detection to the top right of the screen.



Figures 4 and **5** Left is the dark mode, and right is light mode.

In the settings menu, some changes were made to make this screen more user friendly. First of all, the size of the font was increased to improve readability for our older users. We also increased the distance between different options, so that users would no longer accidentally click on the wrong button. Another new addition is the light mode option, since some users expressed an interest in it.



Figures 6 and **7:** Left is the dark mode, and right is light mode for settings

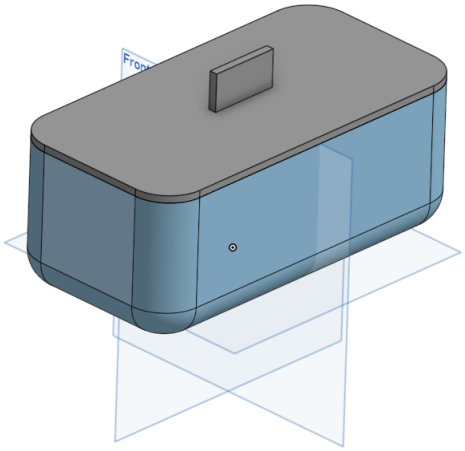
Lessons Learned:

Throughout our project, we have encountered several challenges which we have learned how to resolve. From user testing, in particular, we have learned that it may not be simple to make our device mobile such that it attaches to the phone and is user friendly. Based on our tests, it seems that the users expected the device to be smaller than the phone in order for it to be attached to the phone. We have also learned about the variety of user preferences when it comes to digital design. Some users prefer darker backgrounds with lighter text, but other users are having difficulty seeing the text with this type of design.

We have gone through several experiments with the physical design, specifically the container, which would be the source of user experience with the device. We first built the device from thick watercolor paper as the initial low-fidelity prototype. For our prototype, we used a phone holder which is meant to attach to cars' interior. We added a small lever to our container which the phone holder would attach to. The design we built from watercolor paper was not sturdy enough to balance the weight of the phone holder and phone properly. We decided to try

3D printing, since it could work as a higher fidelity prototype. Plastic is sturdier than paper, so it would have held the phone holder with the phone.

The 3D printing process ended up being slightly more complicated than we initially believed. We learned how to 3D print at WPI. First, we had to go through the training for the WPI makerspace, otherwise we would not be allowed to use the 3D printers. Then, we had to use a CAD program to create our design. We used the free version of Onshape. We used a tutorial for making a box with a clip-on cover, and adapted it for our own purposes. Learning how to work in Onshape was another lesson we learned (see figures 8 and 9).



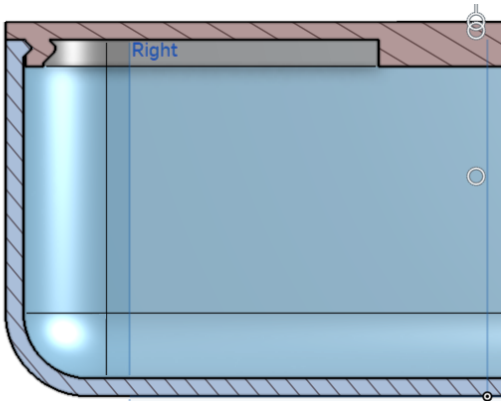
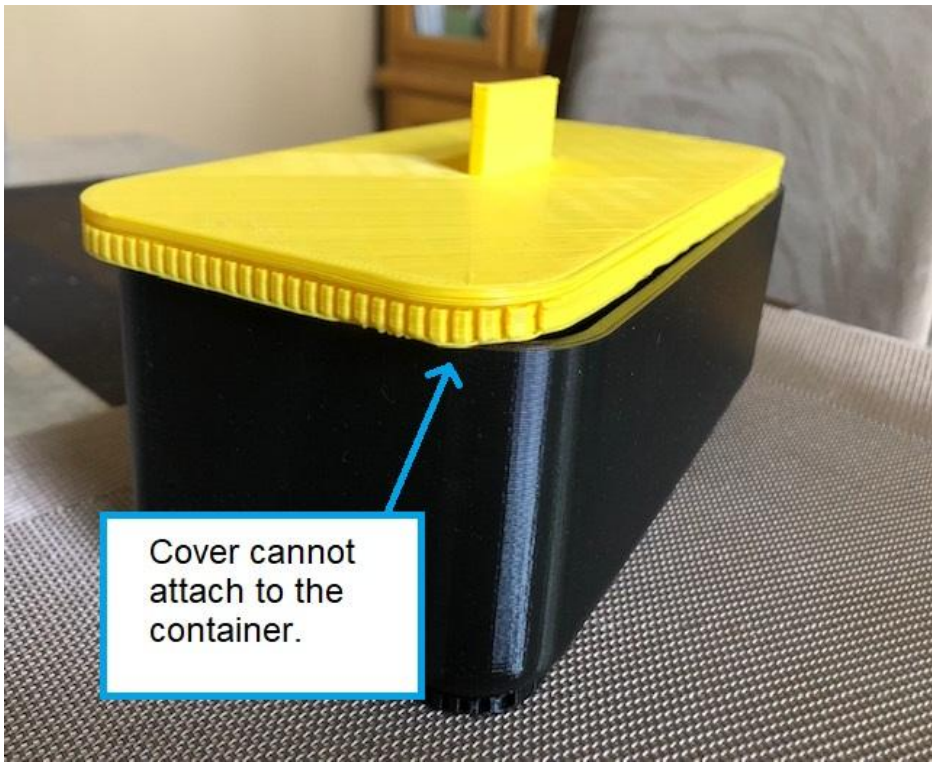


Figure 8 and **9:** The figure at the top shows the CAD design of the container in Onshape. The figure below depicts the mechanisms used to attach the cover to the container. There is a little wedge which is meant to clip the cover onto the box.

WPI uses 3DPrinterOS to connect to campus printers and create the design. We learned about the entire process: loading the correct files onto 3DPrinterOS, converting the dimensions to the correct measurements (from mm to inches), queueing the project, paying for the items at the front desk of Innovation Studios, getting the items approved, and receiving the print. We decided to print the container and cover separately. One big problem we ran into was that if the prints would have a fine render, then they would exceed the maximum allowed printing time (which is 13 hours per item). We learned how to render the prints such that they would take less time to print. Unfortunately, since the render we used did not match fine detail, while the design

of our product depended on fine mechanisms to fit the cover to the box, our 3D printed design failed (see figure 11). The final printed version of the cover could not fit the box. From this, we learned that pieces with important detailed design should be rendered with finer detail in mind.





Figures 10 and **11** The top image shows how the detail did not render correctly on the cover. The bottom image shows how that prevents the cover from clipping into the box.

We also learned that it is important to have a back-up plan, which we did. While working with the 3D prints, we remade the prototype out of cardboard. The cardboard prototype was sturdier than the watercolor paper one, so we ended up using it in our user tests (see figure 12).



Figure 12 The cardboard box prototype we used in our testing. The prototype was painted black.

Using a powerpoint to mimic a digital application was a lot more challenging than we first realized. We used links in the slides to mimic buttons that the user could press in the application. So, if a test user pressed a button, there would be a link that would send them to a different screen, or a slight variation of the same screen. This was very time consuming to implement, and it took 87 slides and hundreds of links to complete. Also, since powerpoints are in landscape view, and the digital prototype itself is in portrait mode, the prototype users tested was much smaller than it should be. A paper prototype may have been a lot easier to make, and would have had less difficulties, but on the other hand, the users were able to interact with this digital prototype almost as if it was the actual application itself.

Bluetooth Module Set Up!

This project also attempted to achieve a high fidelity prototype in terms of actual functionality. Many obstacles have been encountered during the endeavor. Initially, the plan was to use the HC-05 Bluetooth modules. The decision was made to use these modules specifically, because they were the cheapest option that also seemed to be able to provide signal strength information. However, after obtaining the HC-05 modules, it was found that they seemed to have some sort of manufacturing error in their firmware. The modules were supposed to enter a so-called "AT Command Mode" where one might issue commands such as a scan for bluetooth devices. However, it was found that on our particular HC-05 modules, the actual function "AC+INQ", which supposedly instructs the module to perform such a scan, did not seem to get executed. Interestingly, other AT commands, which we did not really need for this project, could get

executed. After playing around with the HC-05 modules for several weeks, eventually we gave up on them in favor of the more robust ESP-32 modules.

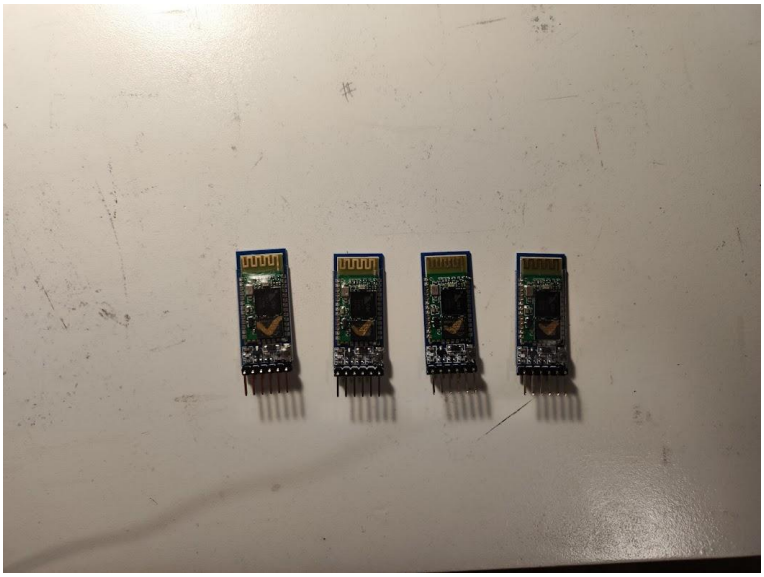


Figure 13. HC-05 Modules

The ESP-32 modules were much more reliable, easier to supply power to and also easier to maintain code for, as they did not require an Arduino, but only the Arduino IDE. Initially we wanted to connect these modules to the phone via cable, by first connecting them to a common USB hub, and then connecting that hub to the phone. However due to a lack of hardware, we opted for a Bluetooth connection between the phone and the modules. Then, in order to power the modules, we found that the best solution was to use a power bank.

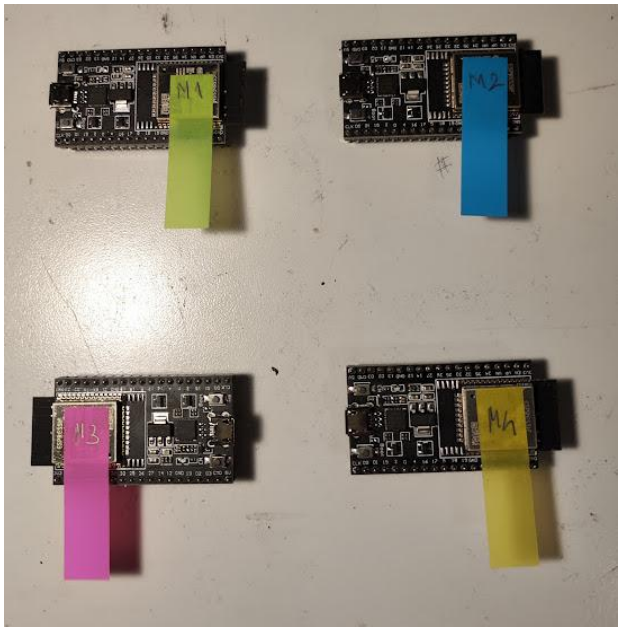


Figure 14. ESP-32 Modules

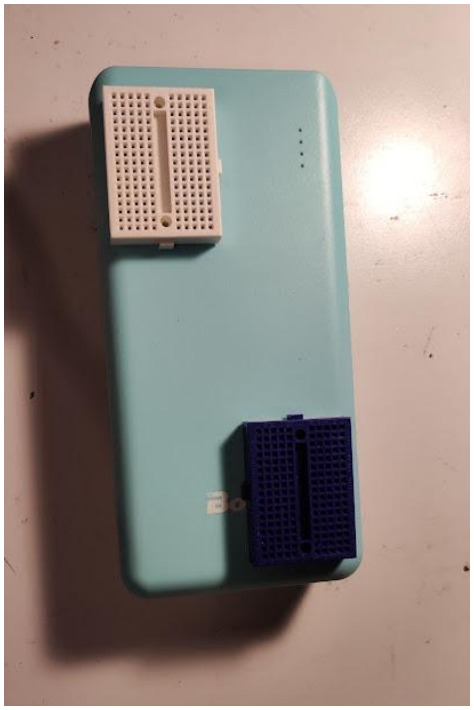


Figure 15. Power Bank used to Power the ESP-32 Modules, with breadboards stuck to its sides which secure the modules in place

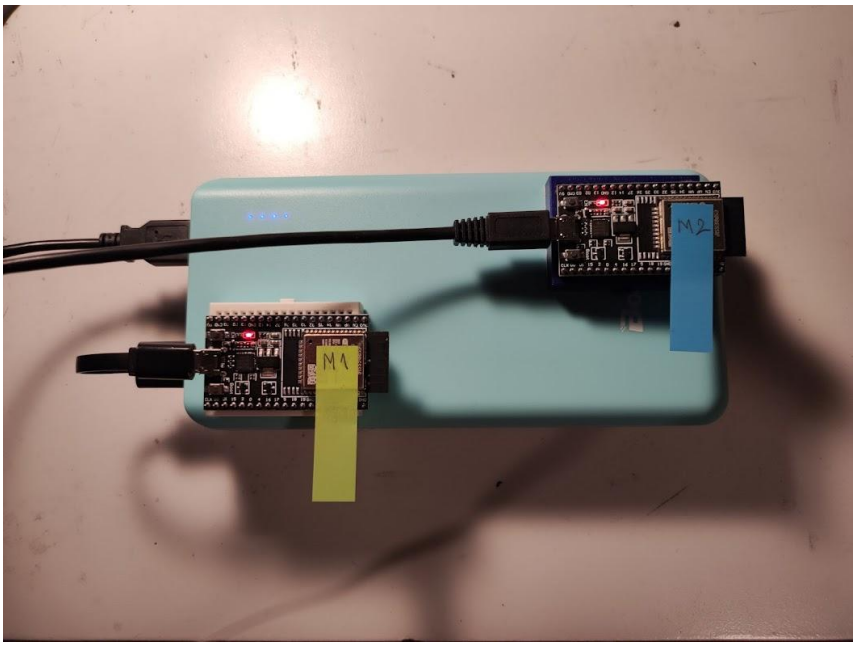


Figure 16. All electronic components connected, acting as 1 whole system

The next issue we ran into was creating a simple one-way communication between the Bluetooth modules and the Android device. Although some online resources existed for connecting 2 Android phones via Bluetooth, no such resources existed for connecting the ESP-32 module.

One of the resources provided kept not working for the ESP-32 module, even though it seemed to have everything in the right place. Eventually, it was figured out that the UUID which the ESP-32 modules were using couldn't be changed and had to be inspected manually using the phone. Once that was done we could finally send information between an ESP-32 module and the phone.

The next obstacle we ran into while using the ESP-32 modules was a lack of resources for how to connect multiple Bluetooth devices to a single phone. This issue was eventually resolved by modifying the pre-existing code and making it run multiple so-called "Bluetooth Connection" threads for each Bluetooth device.

The final obstacle that was encountered was sending signal strength information via Bluetooth to the device. Although we could send simple messages, what we really wanted to do was send information which encoded higher-level programming language objects, so that the same objects could be reconstructed in the phone. We chose JSON as the encoding of objects since we were most familiar with it. The main issue, however, was that the ESP-32 modules used C++, which lacked functionality for converting objects into JSON. Eventually, we decided to construct the JSON object manually with string concatenation which was very error prone and took a lot of time to get right. Finally we were able to send object information between the modules and the phone. The bulk of the remaining work was now left to the phone application, i.e. the sorting and processing of data, and device position triangulation. Due to time restrictions, these were sadly not implemented, but we plan on working on them in the future!

Team Member Contributions:

The contributions for the project were evenly divided upon the members. All members contributed writing and editing of the reports, the presentation, and user testing. The prototype was split among the team members. Jessica Forrett worked on the digital prototype slides, Ivan Martinovic worked on the Bluetooth modules set up and detection (interior technology), and Anna Cherkinsky worked on the design of the physical prototype (container holding the interior).